

# Angular Moderno

## 27 - Formulário de Checkout

---

# 27 — Formulário de Checkout

Integração com carrinho e simulação de compra

**Objetivo da aula:** Integrar o formulário de checkout com o estado global do carrinho e simular uma compra completa.

## Contexto Atual

- Carrinho já funciona com Signals
- Formulário já existe com validações
- Ainda não existe finalização de compra

## Problema

O checkout ainda não executa um fluxo real de compra. Precisamos conectar todas as peças para que a simulação funcione de ponta a ponta.



# Etapa 0: Parar o Servidor

⚠ Antes de qualquer alteração em múltiplos arquivos, pare o servidor para evitar erros de compilação.

No terminal, execute o atalho de teclado:

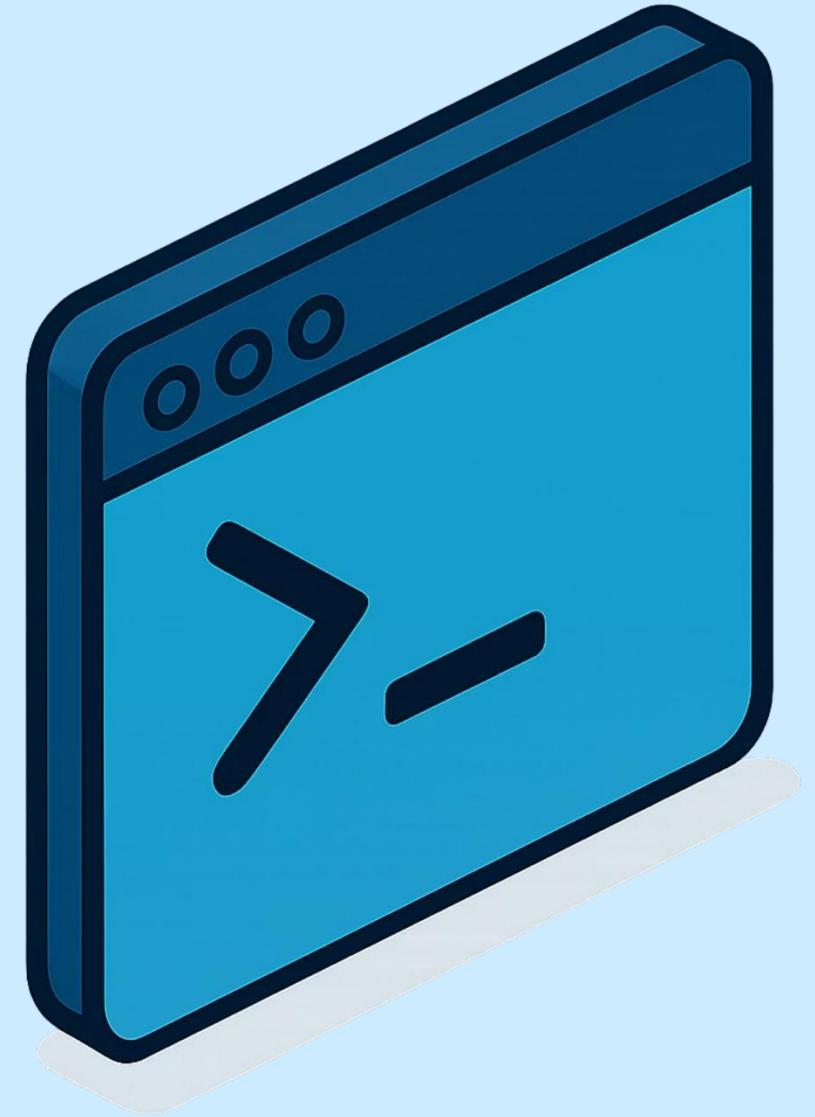
**CTRL + C**

Por que parar?

Evitar erros durante alterações em múltiplos arquivos e garantir compilação correta ao reiniciar.

Quando reiniciar?

Após concluir todas as alterações nos arquivos de serviço e componente de checkout.



# Etapa 1: Ajuste no CarrinhoService

CARRINHO.SERVICE.TS

**Arquivo:** `src/app/core/services/carrinho.service.ts`

Na seção de **SELECTORS**, adicione o seguinte computed:

```
carrinhoVazio = computed(() => this.carrinho().length === 0);
```

O que é um computed?

Um `computed` é um valor derivado de um ou mais signals. Ele recalcula automaticamente sempre que o signal de origem muda.

Para que serve aqui?

- Verificar facilmente se o carrinho está vazio
- Será usado no checkout para impedir finalização inválida
- Evita lógica duplicada em múltiplos componentes

# Etapa 2: Formatando um Item do Carrinho

CARRINHO.SERVICE.TS

**Arquivo:** src/app/core/services/carrinho.service.ts

Logo após os `imports`, insira:

```
type ItemCarrinho = {  
  nome: string;  
  preco: number;  
};
```

SUBSTITUA o `signal` de carrinho por este:

```
private carrinho = signal<ItemCarrinho[]>([]);
```

SUBSTITUA o método `adicionar()` por este:

```
adicionar(produto: ItemCarrinho) {  
  this.carrinho.update((lista) => [...lista, produto]);  
}
```

# Etapa 3: Ajuste no Checkout (Estrutura)

CHECKOUT.TS

**Arquivo:** `src/app/features/checkout/checkout/checkout.ts`

## 1. Ajustar import do Angular

### ANTES

```
import { Component, inject } from '@angular/core';
```

### DEPOIS

```
import { Component, inject, signal } from '@angular/core';
```

## 2. Criar estado de controle

Dentro da classe, adicionar o seguinte signal:

```
compraFinalizada = signal(false);
```

 Esse signal controla a exibição da mensagem de sucesso após finalizar a compra. Começa como `false` e será alterado para `true` ao concluir o fluxo.

# Etapa 4: Lógica de Finalização

CHECKOUT.TS

Arquivo: src/app/features/checkout/checkout/checkout.ts

Substituir o método `finalizar()` pelo código abaixo:

```
finalizar() {  
  this.compraFinalizada.set(false);  
  
  if (this.carrinhoService.carrinhoVazio()) {  
    console.log('Não é possível finalizar uma compra com o carrinho vazio.');    return;  
  }  
  
  if (this.formulario.invalid) {  
    console.log('Formulário inválido');  
    this.formulario.markAllAsTouched();  
    return;  
  }  
  
  const dados = this.formulario.value;  
  const itens = this.carrinhoService.itens();  
  const total = this.carrinhoService.total();  
  
  console.log('Compra finalizada com sucesso!');  
  console.log('Dados do formulário:', dados);  
  console.log('Itens do carrinho:', itens);  
  console.log('Total da compra:', total);  
  
  this.carrinhoService.limpar();  
  this.formulario.reset();  
  this.compraFinalizada.set(true);  
}
```

Carrinho vazio  
Impede finalização sem itens

Valida formulário  
Marca campos inválidos

Simula compra  
Loga dados no console

Limpa estado  
Reseta carrinho e formulário

Sucesso  
Exibe mensagem ao usuário

# Etapa 5: Mensagem de Sucesso

CHECKOUT.HTML

**Arquivo:** `src/app/features/checkout/checkout/checkout.html`

Adicionar logo abaixo do título `<h2>Checkout</h2>` do template:

```
@if (compraFinalizada()) {  
  <p style="color: green">  
    Compra finalizada com sucesso!  
  </p>  
}
```

- ✔ Esse bloco exibe um feedback visual imediato ao usuário após a finalização bem-sucedida da compra. O bloco só aparece quando o sinal `compraFinalizada` for `true`.

# Etapa 6: Carrinho Vazio ou Resumo do Pedido

CHECKOUT.HTML

Arquivo: src/app/features/checkout/checkout/checkout.html

Adicionar **antes do formulário** no template:

```
@if (carrinhoService.carrinhoVazio()) {  
  <p style="color: red">  
    Seu carrinho está vazio. Adicione produtos antes de finalizar a compra.  
  </p>  
} @else {  
  <section>  
    <h3>Resumo do pedido</h3>  
  
    @for (item of carrinhoService.itens(); track $index) {  
      <p>{{ item.nome }} - R$ {{ item.preco.toFixed(2) }}</p>  
    }  
  
    <h3>Total: R$ {{ carrinhoService.total().toFixed(2) }}</h3>  
  </section>  
}
```

Como o bloqueio funciona?

- Usa o `computed` criado no `CarrinhoService`
- Exibe mensagem em vermelho quando o carrinho está vazio
- Desaparece automaticamente ao adicionar itens

Resumo do pedido:



Lista de Itens

Itera sobre os itens do carrinho usando `@for` e exibe nome e preço formatado de cada produto.



Total da Compra

Exibe o total calculado pelo `CarrinhoService`, formatado com duas casas decimais usando `toFixed(2)`.



## Etapa 7: Executar o Projeto

Com todas as alterações salvas, reinicie o servidor de desenvolvimento:

```
ng serve
```

 Acesse a aplicação no navegador em: **`http://localhost:4200/checkout`**

### Servidor iniciado

Aguarde a mensagem de compilação bem-sucedida no terminal antes de acessar o navegador.

### Rota de checkout

Acesse diretamente `/checkout` para testar o fluxo completo de finalização de compra.

### Console do navegador

Abra o DevTools (F12) para visualizar os logs gerados durante a simulação da compra.

# Teste Completo

Fluxo de validação passo a passo

01

Acessar checkout vazio

Abra `/checkout` sem nenhum item no carrinho e veja a mensagem de erro em vermelho.

02

Adicionar produtos

Navegue até a página de produtos, adicione itens ao carrinho e observe o contador atualizar no header.

03

Verificar resumo

Volte para o checkout e confirme que o resumo do pedido exibe os itens e o total corretamente.

04

Preencher formulário

Preencha todos os campos obrigatórios do formulário de checkout com dados válidos.

05

Finalizar e verificar

Clique em finalizar, veja os logs no console, o carrinho limpo e a mensagem de sucesso exibida. Após finalizar, verifique também se o contador do carrinho no header volta para zero.

# Arquivo Completo: CarrinhoService.ts

**Arquivo:** src/app/core/services/carrinho.service.ts

```
import { Injectable, signal, computed } from '@angular/core';

type ItemCarrinho = {
  nome: string;
  preco: number;
};

@Injectable({
  providedIn: 'root',
})
export class CarrinhoService {

  // STATE (GLOBAL)
  private carrinho = signal<ItemCarrinho[]>([]);

  // SELECTORS
  itens = computed(() => this.carrinho());

  quantidade = computed(() => this.carrinho().length);

  total = computed(() =>
    this.carrinho().reduce((total, item) => total + item.preco, 0)
  );

  carrinhoVazio = computed(() => this.carrinho().length === 0);
```

```
// ACTIONS
adicionar(produto: ItemCarrinho) {
  this.carrinho.update(lista => [...lista, produto]);
}

limpar() {
  this.carrinho.set([]);
}
}
```

# Arquivo Completo: Checkou.t.ts

**Arquivo:** src/app/features/checkout/checkout/checkout.ts

```
import { Component, inject, signal } from '@angular/core';
import {
  ReactiveFormsModule,
  FormGroup,
  FormControl,
  Validators,
  AbstractControl,
  ValidationErrors,
} from '@angular/forms';
import { CarrinhoService } from '../../../core/services/carrinho.service';

function nomeSemNumeros(control: AbstractControl): ValidationErrors | null {
  const valor = control.value;
  if (!valor) return null;
  if (/\\d/.test(valor)) {
    return { numeroInvalido: true };
  }
  return null;
}

@Component({
  selector: 'app-checkout',
  imports: [ReactiveFormsModule],
  templateUrl: './checkout.html',
  styleUrls: ['./checkout.css'],
})
```

```
export class Checkout {
  carrinhoService = inject(CarrinhoService);
  compraFinalizada = signal(false);

  formulario = new FormGroup({
    nome: new FormControl('', [Validators.required, Validators.minLength(3), nomeSemNumeros]),
    email: new FormControl('', [Validators.required, Validators.email]),
    endereco: new FormControl('', [Validators.required, Validators.minLength(5)]),
  });

  finalizar() {
    this.compraFinalizada.set(false);
    if (this.carrinhoService.carrinhoVazio()) {
      console.log('Não é possível finalizar uma compra com o carrinho vazio.');      return;
    }
    if (this.formulario.invalid) {
      console.log('Formulário inválido');
      this.formulario.markAllAsTouched();
      return;
    }
  }
```

```
    const dados = this.formulario.value;
    const itens = this.carrinhoService.itens();
    const total = this.carrinhoService.total();
    console.log('Compra finalizada com sucesso!');
    console.log('Dados do formulário:', dados);
    console.log('Itens do carrinho:', itens);
    console.log('Total da compra:', total);
    this.carrinhoService.limpar();
    this.formulario.reset();
    this.compraFinalizada.set(true);
  }
}
```

# Arquivo Completo: Checkout HTML

**Arquivo:** src/app/features/checkout/checkout/checkout.html

```
<h2>Checkout</h2>

@if (compraFinalizada()) {
  <p style="color: green">
    Compra finalizada com sucesso!
  </p>
}
@if (carrinhoService.carrinhoVazio()) {
  <p style="color: red">
    Seu carrinho está vazio. Adicione produtos antes de finalizar a compra.
  </p>
} @else {
  <section>
    <h3>Resumo do pedido</h3>
    @for (item of carrinhoService.itens(); track $index) {
      <p>{{ item.nome }} - R$ {{ item.preco.toFixed(2) }}</p>
    }
    <h3>Total: R$ {{ carrinhoService.total().toFixed(2) }}</h3>
  </section>
}

<form [formGroup]="formulario">
  <p><input type="text" formControlName="nome" placeholder="Nome" /></p>

  @if (formulario.get('nome')?.touched && formulario.get('nome')?.invalid) {
    @if (formulario.get('nome')?.errors?.['required']) {
      <small>Nome é obrigatório</small>
    }
    @if (formulario.get('nome')?.errors?.['minlength']) {
      <small>Nome deve ter no mínimo 3 caracteres</small>
    }
  }
}
```

```
@if (formulario.get('nome')?.errors?.['numeroInvalido']) {
  <small>Nome não pode conter números</small>
}

<p><input type="email" formControlName="email" placeholder="E-mail" /></p>

@if (formulario.get('email')?.touched && formulario.get('email')?.invalid) {
  @if (formulario.get('email')?.errors?.['required']) {
    <small>Email é obrigatório</small>
  }
  @if (formulario.get('email')?.errors?.['email']) {
    <small>Email inválido</small>
  }
}

<p>
  <input type="text" formControlName="endereco" placeholder="Endereço" />
</p>

@if (formulario.get('endereco')?.touched && formulario.get('endereco')?.invalid) {
  @if (formulario.get('endereco')?.errors?.['required']) {
    <small>Endereço é obrigatório</small>
  }
  @if (formulario.get('endereco')?.errors?.['minlength']) {
    <small>Endereço deve ter no mínimo 5 caracteres</small>
  }
}

<p>
  <button type="button" (click)="finalizar()" [disabled]="formulario.invalid">
    Finalizar Compra
  </button>
</p>
</form>
```

# Resultado da Aula



## Checkout Integrado

Checkout totalmente integrado ao estado global do carrinho via Signals.



## Simulação Funcional

Simulação de compra completa com logs detalhados no console do navegador.



## Validação de Carrinho

Bloqueio lógico da finalização quando o carrinho está vazio, com feedback visual no checkout.



## Limpeza de Estado

Carrinho e formulário são resetados automaticamente após a compra.



## Feedback Visual

Mensagem de sucesso exibida ao usuário após finalização bem-sucedida.

 **Importante:** A aplicação ainda não possui autenticação nem backend real. Esses pontos serão implementados nas próximas aulas.

Dúvidas

?

